INSTRUCTOR ANSWER KEY & MARKING SCHEME

Course: Introduction to C# & .NET Framework Programming | Duration: 180 Mins | Total Marks: 100

Grading Rubric & Score Distribution Overview:
• **Part I (Q1–Q30):** 30 Multiple Choice Questions (1 Mark each = 30 Marks). Exact key match required.
• **Part II (Q31–Q50):** 20 Short Answer & Conceptual Essay Questions (1.5 Marks each = 30 Marks). Award partial credit based on key terms.
• **Part III (Q51–Q80):** 30 Code Implementation Tasks (1 Mark each = 30 Marks). Evaluate syntax, correctness, and logical completeness.
• **Part IV (Q81–Q100):** 20 Code Debugging & Correction Tasks (0.5 Marks each = 10 Marks). Identify defect and provide working fix.

PART I: MULTIPLE CHOICE QUESTION ANSWER KEY (30 MARKS)

Each question is worth 1.0 Mark. Total: 30 Marks.

Q #	Ans	Topic & Rationale	Q #	Ans	Topic & Rationale
Q1	A	.NET Framework executes code via Common Language Runtime (CLR).	Q16	A	while loop evaluates condition prior to executing body.
Q2	B	C# source code compiles into Microsoft Intermediate Language (MSIL).	Q17	B	do-while loop guarantees at least one execution of body before checking condition.
Q3	B	Visual Studio Community is an Integrated Development Environment (IDE).	Q18	A	for loop header contains initializer, condition, and iterator.
Q4	B	int is a 32-bit signed integer value type in .NET.	Q19	A	break statement immediately terminates execution of loop or switch.
Q5	A	Implicit conversion occurs safely without casting when no data loss occurs.	Q20	B	continue statement skips current iteration and moves to next loop cycle.
Q6	B	Explicit cast (int)3.87 truncates decimal portion to 3.	Q21	B	Arrays in C# are zero-indexed fixed-size collection structures.
Q7	B	System.Console class provides ReadLine() for user keyboard input.	Q22	A	Accessing invalid array index throws IndexOutOfRangeException.
Q8	A	Console.ReadLine() always returns data as a System.String.	Q23	A	List is a dynamic generic collection residing in System.Collections.Generic.
Q9	A	int.Parse("123") successfully converts valid numeric string to int 123.	Q24	A	List.Add() appends new element to end of dynamic list.
Q10	B	int.TryParse returns boolean indicating whether parsing succeeded.	Q25	B	Value types store data directly on Stack; Reference types reside on Heap.
Q11	A	% is the modulus operator returning the remainder of integer division.	Q26	A	class defines reference type blueprint containing fields, properties, and methods.
Q12	B	Integer division 7 / 2 truncates fractional part, yielding 3.	Q27	A	private modifier restricts field visibility strictly to declaring class.
Q13	A	&& operator short-circuits evaluation if first operand is false.	Q28	B	Constructors share declaring class name and lack return type specification.

Q14	**A**	if-else statement chooses between execution branches based on condition.	**Q29**	**A**	System.IO namespace houses StreamReader and StreamWriter file handling classes.
Q15	**B**	switch statements evaluate expressions against discrete constant values.	**Q30**	**B**	StreamReader.ReadLine() returns null when reaching End of File (EOF).

PART II: SHORT ANSWER & CONCEPTUAL ESSAY RUBRIC (30 MARKS)

Each question is worth 1.5 Marks. Award full marks if core concepts are present.

Q#	Topic	Core Model Answer & Grading Criteria	Pts
Q31	**.NET Compilation Lifecycle**	C# code compiles into MSIL (Intermediate Language). At runtime, the JIT (Just-In-Time) compiler translates MSIL into native CPU machine instructions.	**1.5**
Q32	**CLR Responsibilities**	Provides managed environment: memory management (Garbage Collection), type safety, thread execution, exception handling, and security enforcement.	**1.5**
Q33	**Value Types vs Reference Types**	Value types (int, double, bool, struct) store data directly on Stack. Reference types (string, class, array) store reference on Stack pointing to object on Heap.	**1.5**
Q34	**Implicit vs Explicit Casting**	Implicit: automatic safe conversion from smaller to larger types (int -> double). Explicit: manual cast required when data loss is possible e.g. (int)3.14.	**1.5**
Q35	**int.Parse vs int.TryParse**	int.Parse throws Exception if format is invalid. int.TryParse returns bool indicating success/failure and outputs converted value safely via out parameter.	**1.5**
Q36	**Integer Division vs Floating-Point**	Integer division truncates decimals (7/2 = 3). Floating-point division preserves decimals (7.0/2.0 = 3.5).	**1.5**
Q37	**Short-Circuit Logical Operators**	&& stops evaluating if left operand is false. || stops evaluating if left operand is true. Prevents unnecessary computation and null errors.	**1.5**
Q38	**if-else vs switch Statement**	if-else checks flexible range/boolean conditions. switch compares single expression against discrete constant values efficiently.	**1.5**
Q39	**while vs do-while Loop**	while tests condition before body (may execute 0 times). do-while executes body first before condition (guaranteed >= 1 execution).	**1.5**
Q40	**break vs continue**	break completely exits the enclosing loop or switch. continue skips remainder of current iteration and jumps to next loop condition evaluation.	**1.5**
Q41	**Fixed Array vs Dynamic List**	Array (int[]) has fixed size set upon allocation. List dynamically resizes, providing flexible methods like Add() and Remove().	**1.5**
Q42	**Array Indexing & Bound Checking**	Arrays are zero-indexed (0 to Length - 1). Accessing index < 0 or >= Length throws runtime IndexOutOfRangeException.	**1.5**
Q43	**Stack vs Heap Allocation**	Stack: fast, structured LIFO memory for method calls and local value types. Heap: managed memory pool for dynamic object instances.	**1.5**
Q44	**Garbage Collection (GC)**	Automatic CLR background process that inspects Heap, identifies unreachable/unreferenced objects, and reclaims memory.	**1.5**
Q45	**Class vs Object**	Class is abstract blueprint/type definition. Object is concrete instance created in memory using 'new' keyword.	**1.5**
Q46	**Encapsulation & Access Modifiers**	Hiding internal state by making fields private and restricting access via public methods/properties to protect object integrity.	**1.5**
Q47	**Constructor Purpose**	Special method invoked during instantiation ('new') to initialize object fields and ensure valid initial state.	**1.5**

Q48	**Auto-Implemented Properties**	Concise C# syntax (public int Age { get; set; }) where compiler automatically creates private backing field behind property.	**1.5**
Q49	**StreamReader & StreamWriter**	Classes in System.IO for file handling. StreamReader reads text characters line-by-line; StreamWriter writes strings to text file stream.	**1.5**
Q50	**Exception Handling & try-catch**	try block contains code that might throw runtime error. catch block intercepts exception, preventing application crash and handling error.	**1.5**

PART III: CODE IMPLEMENTATION SOLUTIONS (30 MARKS)

Each task is worth 1.0 Mark. Award credit for working syntax and proper logic.

Q#	Task Title	Correct Implementation / Reference Code	Pts
Q51	**Console Output**	`Console.WriteLine("Welcome to .NET Programming!");`	**1.0**
Q52	**Variable Declaration**	`int age = 20; double gpa = 3.85; string studentName = "Alice";`	**1.0**
Q53	**Input & Parsing**	`string input = Console.ReadLine(); int count = int.Parse(input);`	**1.0**
Q54	**Safe Parsing**	`if (int.TryParse(Console.ReadLine(), out int val)) { Console.WriteLine(val); }`	**1.0**
Q55	**Explicit Casting**	`double val = 9.88; int result = (int)val;`	**1.0**
Q56	**If-Else Even/Odd**	`if (num % 2 == 0) Console.WriteLine("Even"); else Console.WriteLine("Odd");`	**1.0**
Q57	**Ternary Operator**	`string status = (score >= 50) ? "Pass" : "Fail";`	**1.0**
Q58	**Switch Grade**	`switch(grade) { case 'A': Console.WriteLine("Excellent"); break; default: Console.WriteLine("Valid"); break; }`	**1.0**
Q59	**For Loop 1 to 10**	`for (int i = 1; i <= 10; i++) { Console.WriteLine(i); }`	**1.0**
Q60	**While Loop Sum**	`int sum = 0, i = 1; while(i <= 5) { sum += i; i++; }`	**1.0**
Q61	**Do-While Positive**	`int num; do { num = int.Parse(Console.ReadLine()); } while(num <= 0);`	**1.0**
Q62	**Array Initialization**	`int[] numbers = new int[] { 10, 20, 30, 40, 50 };`	**1.0**
Q63	**Array Sum Iteration**	`int total = 0; foreach(int n in numbers) { total += n; }`	**1.0**
Q64	**Array Max Value**	`int max = arr[0]; foreach(int x in arr) { if(x > max) max = x; }`	**1.0**
Q65	**List Creation**	`List cities = new List() { "Helsinki", "Espoo" };`	**1.0**
Q66	**List Filtering**	`foreach(string c in cities) { if(c.Length > 5) Console.WriteLine(c); }`	**1.0**

Q67	**Class Blueprint**	`class Student { public string Name; public int Id; }`	**1.0**
Q68	**Constructor**	`public Student(string name, int id) { Name = name; Id = id; }`	**1.0**
Q69	**Auto Property**	`public double GPA { get; set; }`	**1.0**
Q70	**Private Field Guard**	`private int age; public void SetAge(a) { if(a > 0) age = a; }`	**1.0**
Q71	**Object Instantiation**	`Student s1 = new Student("Bob", 101);`	**1.0**
Q72	**Method Invocation**	`s1.DisplayDetails();`	**1.0**
Q73	**File Line Reader**	`using(StreamReader sr = new StreamReader("test.txt")) { string line = sr.ReadLine(); }`	**1.0**
Q74	**File Loop Reader**	`using(StreamReader sr = new StreamReader("data.txt")) { string l; while((l = sr.ReadLine()) != null) Console.WriteLine(l); }`	**1.0**
Q75	**File Writer**	`using(StreamWriter sw = new StreamWriter("out.txt")) { sw.WriteLine("Passed Exam"); }`	**1.0**
Q76	**Try-Catch Division**	`try { int res = a / b; } catch(DivideByZeroException ex) { Console.WriteLine("Error"); }`	**1.0**
Q77	**Array Reversal**	`Array.Reverse(arr);`	**1.0**
Q78	**List Remove Element**	`list.RemoveAll(x => x < 0);`	**1.0**
Q79	**String Format**	`string formatted = $"Student {Name} scored {Score}";`	**1.0**
Q80	**Count Matching List**	`int count = 0; foreach(var item in items) if(item > 10) count++;`	**1.0**

PART IV: CODE DEBUGGING & CORRECTION SCHEME (10 MARKS)

Each item is worth 0.5 Marks. Award full points for identifying bug and writing fix.

Q#	Bug / Defect Description	Corrected Code Snippet	Pts
Q81	**Err:** int x = "10"; **Cause:** Type mismatch: string assigned to int.	`int x = int.Parse("10");`	**0.5**
Q82	**Err:** int[] a = new int[3]; a[3] = 5; **Cause:** IndexOutOfRangeException: valid index 0..2.	`a[2] = 5;`	**0.5**
Q83	**Err:** double avg = 5 / 2; **Cause:** Integer division truncates result to 2.0.	`double avg = 5.0 / 2;`	**0.5**
Q84	**Err:** if (x = 5) { } **Cause:** Assignment '=' used instead of equality '=='.	`if (x == 5) { }`	**0.5**
Q85	**Err:** string s; Console.WriteLine(s.Length); **Cause:** Unassigned local variable 's'.	`string s = ""; Console.WriteLine(s.Length);`	**0.5**

Q86	**Err:** while(i < 10) { Console.WriteLine(i); } **Cause:** Infinite loop: variable 'i' never incremented.	`while(i < 10) { Console.WriteLine(i); i++; }`	**0.5**
Q87	**Err:** List lst; lst.Add(5); **Cause:** NullReferenceException: list not instantiated.	`List lst = new List(); lst.Add(5);`	**0.5**
Q88	**Err:** for(int i=0; i<=arr.Length; i++) **Cause:** Index out of bounds when i == arr.Length.	`for(int i=0; i`	**0.5**
Q89	**Err:** StreamReader sr = new StreamReader("f.txt"); **Cause:** Resource leak: stream file lock not closed.	`using(StreamReader sr = new StreamReader("f.txt"))`	**0.5**
Q90	**Err:** void Main() { return 0; } **Cause:** Void method returning integer value.	`int Main() { return 0; }`	**0.5**
Q91	**Err:** public class S { int age; } s.age = 20; **Cause:** Field 'age' is private by default.	`public class S { public int age; }`	**0.5**
Q92	**Err:** int x = null; **Cause:** Value type 'int' cannot be assigned null.	`int? x = null; OR int x = 0;`	**0.5**
Q93	**Err:** switch(x) { case 1: y=1; case 2: y=2; } **Cause:** Fall-through error: missing break statement.	`case 1: y=1; break;`	**0.5**
Q94	**Err:** string s = 'Hello'; **Cause:** Single quotes used for string literal.	`string s = "Hello";`	**0.5**
Q95	**Err:** int[] arr = new int[]; **Cause:** Array allocation missing size dimension.	`int[] arr = new int[5];`	**0.5**
Q96	**Err:** const int x; x = 10; **Cause:** Const field must be initialized at declaration.	`const int x = 10;`	**0.5**
Q97	**Err:** if(a > 0) && (b > 0) **Cause:** Syntax error: parenthesis grouping invalid.	`if (a > 0 && b > 0)`	**0.5**
Q98	**Err:** Convert.ToInt32(null) throws crash **Cause:** Misconception: Convert.ToInt32(null) returns 0.	`Returns 0 safely (No error).`	**0.5**
Q99	**Err:** List l = new List(); **Cause:** Generic type argument missing on instantiation.	`List l = new List();`	**0.5**
Q100	**Err:** static void Run() { this.Display(); } **Cause:** Static method cannot reference 'this' instance.	`Remove static OR pass instance reference.`	**0.5**

SUMMARY GRADING SCALE & CONVERSION TABLE

Score Range (%)	Letter Grade	Performance Level Description
90% - 100%	**A / Outstanding**	Mastery of .NET Framework, C# language syntax, memory model, and file handling.
80% - 89%	**B / Very Good**	Solid proficiency in core programming, OOP design, collections, and control logic.
70% - 79%	**C / Satisfactory**	Sufficient knowledge of basic syntax and loops; minor errors in OOP or file I/O.
60% - 69%	**D / Pass**	Meets minimum course requirements; struggles with memory allocation and debugging.
< 60%	**F / Fail**	Insufficient grasp of foundational C# programming and .NET architecture.